PROCESS REINFORCEMENT THROUGH IMPLICIT REWARDS

Ganqu Cui[2,1†*], **Lifan Yuan**[3†*], **Zefan Wang**[1*], **Hanbin Wang**[4*], **Wendi Li**[1*], **Bingxiang He**[1*], **Yuchen Fan**[2,5*], **Tianyu Yu**[1*], **Qixin Xu**[1*], **Weize Chen**[1], **Jiarui Yuan**[1], **Huayu Chen**[1], **Kaiyan Zhang**[1], **Xingtai Lv**[1], **Shuo Wang**[1], **Yuan Yao**[1], **Xu Han**[1], **Hao Peng**[3], **Yu Cheng**[2,6], **Zhiyuan Liu**[1], **Maosong Sun**[1], **Bowen Zhou**[2,1], **Ning Ding**[1†]
[1]Tsinghua University [2]Shanghai AI Lab [3]University of Illinois Urbana-Champaign
[4]Peking University [5]Shanghai Jiaotong University [6]CUHK
cuiganqu@pjlab.org.cn lifan4@illinois.edu

https://github.com/PRIME-RL/PRIME

ABSTRACT

Dense process rewards have proven a more effective alternative to the sparse outcome-level rewards in the inference-time scaling of large language models (LLMs), particularly in tasks requiring complex multi-step reasoning. While dense rewards also offer an appealing choice for the reinforcement learning (RL) of LLMs since their fine-grained rewards have the potential to address some inherent issues of outcome rewards, such as training efficiency and credit assignment, this potential remains largely unrealized. This can be primarily attributed to the challenges of training process reward models (PRMs) online, where collecting high-quality process labels is prohibitively expensive, making them particularly vulnerable to reward hacking. To address these challenges, we propose PRIME (**P**rocess **R**einforcement through **IM**plicit r**E**wards), which enables online PRM updates using only policy rollouts and outcome labels through *implict process rewards*. PRIME combines well with various advantage functions and forgoes the dedicated reward model training phase that existing approaches require, substantially reducing the development overhead. We demonstrate PRIME's effectiveness on competitional math and coding. Starting from Qwen2.5-Math-7B-Base, PRIME achieves a 15.1% average improvement across several key reasoning benchmarks over the SFT model. Notably, our resulting model, Eurus-2-7B-PRIME, surpasses Qwen2.5-Math-7B-Instruct on seven reasoning benchmarks with 10% of its training data.[1]

1 INTRODUCTION

Dense process rewards, which provide feedback at each intermediate step rather than only the whole trajectory, have proven effective in inference-time scaling of large language models (LLMs) on challenging reasoning tasks (Uesato et al., 2022; Lightman et al., 2023; Wang et al., 2023; Yuan et al., 2024b). On the training side, they also present superiorities in the reinforcement learning (RL) of LLMs, particularly in improving training efficiency (Sutton & Barto, 2018) and credit assignment (Leike et al., 2018) compared with sparse outcome rewards. However, successful applications of dense rewards in RL for LLMs are limited (Setlur et al., 2024), as current industry-leading models primarily depend on verifiable outcome rewards and have not yet demonstrated meaningful progress with dense rewards (DeepSeek-AI et al., 2025; Team et al., 2025).

We identify the central challenge as *how to acquire and utilize high-quality dense rewards at scale*, which enables online process reward model (PRM) update efficiently. The reason is that, optimizing towards a static reward model eventually leads to overoptimization or reward hacking (Gao et al.,

*Core Contributors.
†Project Lead.
[1]Models and data are available at: https://github.com/PRIME-RL/PRIME.

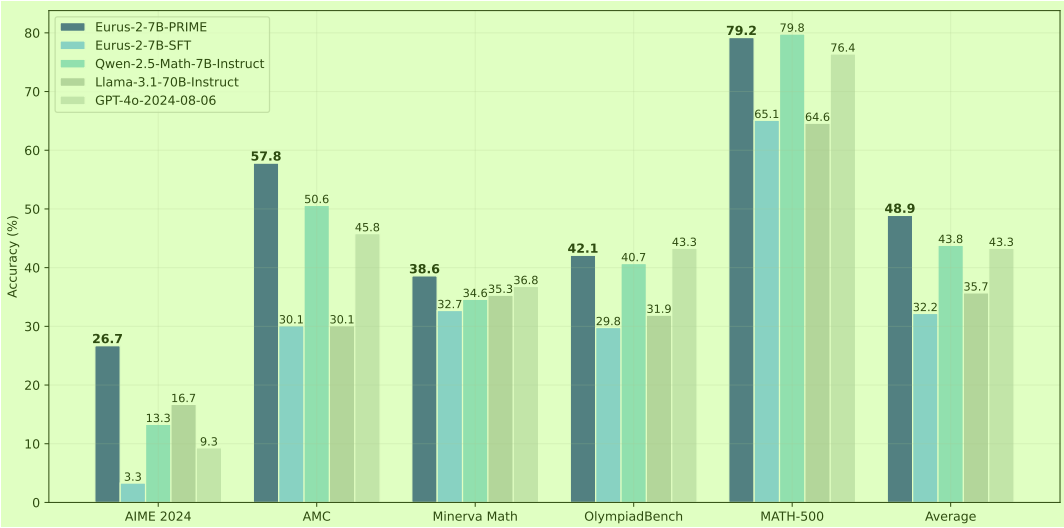


Figure 1: Overall math performance. Eurus-2-7B-PRIME excels at competition-level mathematics benchmarks, outperforming advanced math models and larger models. Notably, PRIME brings substantial performance gain (+16.7%) over Eurus-2-7B-SFT.

2022) due to distribution shift. Ideally, this can be solved by improving the reward model online (Leike et al., 2018). However, acquiring dense process labels for training is prohibitively more expensive. Existing methods either need to build complicated human annotation pipelines (Lightman et al., 2023) or rely on estimation-based methods, which require about 10× more rollouts for each step than sampling only the response-level trajectories (Wang et al., 2023; Kazemnejad et al., 2024). Neither of them is scalable in online RL. Moreover, to the best of our knowledge, it remains underexplored how to incorporate dense rewards into RL for LLMs.

In this work, we propose Process Reinforcement through Implicit Rewards (PRIME), a scalable framework for enhancing reasoning capabilities via efficient reinforcement learning with dense token-level rewards. At its core, the framework employs recently proposed implicit process reward modeling (Yuan et al., 2024b) to train dense reward models with only outcome-level labels. This enables PRIME to perform online learning of reward signals using only outcome labels on policy rollouts, thereby fundamentally mitigating reward hacking while maintaining the same computational cost as traditional outcome reward models (ORMs). Besides scalability, PRIME also (1) serves as a general method to fuse token-level dense rewards and sparse outcome rewards by calculating their returns separately before summing together, which is compatible with diverse RL algorithms (Williams, 1992; Kool et al., 2019; Shao et al., 2024; Ahmadian et al., 2024; Schulman et al., 2017); (2) eliminates the dedicated reward modeling stage, which is required by existing works, by simply initializing from the SFT model or even the base model (§ 5.6). In summary, starting from one single language model, the PRIME framework can efficiently accomplish the generation of dense rewards, the initialization and updating of reward models, as well as the reinforcement learning (RL) training of the policy model.

In experiments, we train Qwen2.5-Math-7B-Base (Yang et al., 2024b) with PRIME after a lightweight SFT warmup stage. Compared to RL using outcome rewards only, PRIME achieves a 2.5× sample efficiency gain and a 6.9% performance improvements on challenging math problems. As shown in Figure 1, through PRIME, we successfully achieve substantial improvement on key mathematical reasoning benchmarks over the SFT model, leading to **16.7%** improvement on average, and over **20%** on AMC&AIME competitions. Our final model Eurus-2-7B-PRIME surpassed Qwen2.5-Math-7B-Instruct on five key mathematical benchmarks. Notably, this is achieved with only 10% of the data used by Qwen-Math, as in Table 1.

Table 1: The comparison of resource requirements between Eurus-2-7B-PRIME and Qwen2.5-Math-7B-Instruct.

Model	Eurus-2-7B-PRIME	Qwen2.5-Math-7B-Instruct
Base Model	Qwen2.5-Math-7B	Qwen2.5-Math-7B
SFT Data	230K (open-source)	2.5M (open-source & in-house)
RM Data	0	618K (in-house)
RM	Eurus-2-7B-SFT	Qwen2.5-Math-RM (72B)
RL Data	150K queries × 4 samples	66K queries × 32 samples

Our analysis shows that updating the PRM online is key to the success of PRIME (§5.1). We also show that PRIME could generally boost various RL algorithms, including RLOO (Ahmadian et al., 2024), REINFORCE (Williams, 1992), PPO (Schulman et al., 2017), and GRPO (Shao et al., 2024) (§5.4). In terms of the design choices of advantage estimate, we observe that Implicit PRMs are better to be used as reward models than value models (§5.5).

2 Reinforcement Learning for LLMs and the Challenges of Incoporating Dense Rewards

Reinforcement Learning (RL) aims to learn an optimal policy π_θ that maximizes the expected cumulative discounted reward, namely return, when interacting with an environment. In the context of autoregressive language modeling, state at step t is the concatenation of prompt $\mathbf{x}$ and current response $\mathbf{y}_{<t}$, and the action is the t-th token or step y_t.

2.1 RL Preliminaries for LLMs

Policy Gradient. Policy gradient is a fundamental algorithm that directly optimizes this objective. Central to this approach is the advantage function A_t, which quantifies how much better an action is compared to alternatives in a given state:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\mathbf{x}\sim\mathcal{D},\mathbf{y}\sim\pi_\theta}\left[\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(y_t|\mathbf{y}_{<t})A_t\right] \tag{1}$$

where $(\mathbf{x}, \mathbf{y})$ represents a pair of input and output. $\mathbf{x}$ is omitted for brevity. In practice, the advantage function is implemented as cumulative discounted rewards subtracting a baseline:

$$A_t = \sum_{s=t}^{T}\gamma^{s-t}r(y_s) - b \tag{2}$$

$\gamma \in [0, 1]$ is a discount factor that optionally decays future rewards, and $r(y_s)$ is the reward provided by the environment at time step s with x and $\mathbf{y}_{<s}$ being omitted in conditions. Eq. 2 is the general formula of the Monte-Carlo (MC) advantage estimate, which indicates that, the high-quality and dense reward at each step is crucial for RL. Different choices of b include, e.g. directly using values Williams (1992), group average of rewards (Shao et al., 2024), and leave-one-out average of rewards Ahmadian et al. (2024); Kool et al. (2019).

Value Models. Though the MC estimate is unbiased, it suffers from high variance because of the reliance on all future actions and rewards, which can be random and noisy. Value models, which predict expected accumulated rewards starting from a state, are adopted to help reduce the variance in advantage estimation, such as Generalized Advantage Estimation (GAE; Schulman et al., 2016): $A_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{s=0}^{\infty}(\gamma\lambda)^s\delta_{t+s}$, where $\delta_t = r(y_t) + \gamma V(\mathbf{y}_{<t+1}) - V(\mathbf{y}_{<t})$ is the temporal difference (TD) error (Sutton, 1988), V is a value model, and λ controls the bias-variance tradeoff in advantage estimation. PPO (Schulman et al., 2017) is a representative of such actor-critic algorithms that explicitly train a value model along with the policy.

Reward Sparsity. Although dense rewards can be naturally integrated into the advantage function through Eq. 2, unfortunately, only outcome reward models (ORMs) are available in most practices of LLMs, i.e., only the final token bears a meaningful reward while intermediate tokens receive no rewards (Rafailov et al., 2023; Shao et al., 2024; DeepSeek-AI et al., 2025). In this bandit setting, $r(y_t) = 0$ for $t < T$ while $r(y_T)$ can be non-zero, and Eq. 2 becomes $A = r(y_T) - b$. This formulation, while simpler, can suffer from reward sparsity issues as the policy receives feedback only at the end of the entire generation. This may (1) encourage spurious solutions with incorrect processes but correct answers, (2) largely reduce sample efficiency in training, and (3) encounter the credit assignment problem (Sutton & Barto, 2018). These drawbacks could be further amplified on complicated tasks, which require more thinking and execution steps, urging the need of dense rewards (Uesato et al., 2022; Lightman et al., 2023). Some may consider employing a value model to mitigate the problem, as it predicts values at every step t. However, previous work showed that value models may not be able to solve the reward sparsity issue effectively due to training challenges, despite the additional computation overhead (Shao et al., 2024; Ahmadian et al., 2024). We will also empirically validate this claim in §5.5.

2.2 KEY CHALLENGES IN SCALABLE DENSE REWARDS

The way to mitigate the reward sparsity problem is to adopt dense reward models, namely PRMs, which score model responses over each token or step. However, it is usually infeasible in practice to incorporate dense rewards into online RL because of three critical challenges in implementation.

C1. Process rewards are hard to define. It is difficult to collect step-level labels since reasoning steps do not naturally occur in sequences. Although tokens are easily distinguishable, annotating labels for each token is too costly. Moreover, defining the absolute correctness of intermediate processes as dense rewards can be ambiguous, as some incorrect steps can also positively contribute to the final answer by pruning searching branches (OpenAI, 2024; DeepSeek-AI et al., 2025).

C2. PRM online updates are not scalable. It is crucial to prevent reward overoptimization or reward hacking, which requires the reward model or value model to be updated online along with the policy model (Schulman et al., 2017; Gao et al., 2022). However, training PRMs often requires extensive nuanced step-level annotation, which is infeasible in online RL training. Therefore, this brings about considerable scalability and generalization concerns in dense rewards for RL.

C3. Explicit reward modeling brings extra cost. Training reward models requires extensive annotation and broad data coverage to ensure a good balance between adaptability to the policy distribution and generalization to distribution shifts. Hence, the explicit training stage introduces a very costly data collection and an additional training overhead, especially for PRMs which typically require stepwise labels.

Notably, a concurrent work shares similar conclusions and thus is impeded from incorporating PRMs into their large-scale RL training (DeepSeek-AI et al., 2025).

3 PRIME

To address the above challenges, we propose PRIME, a scalable online RL method with dense rewards. The key insight of PRIME is to apply *implicit process rewards*, which are derivable from the Implicit PRM that is trained with only outcome labels (Yuan et al., 2024b). This property enables us to update the PRMs online to avoid reward hacking. We then design a flexible framework to incorporate implicit process rewards with outcome rewards into any kind of MC advantage estimate. PRIME is illustrated in Figure 2 and Algorithm 1. Next, we will detail the implicit process rewards (§3.1) and how we leverage them to calculate advantages (§3.2), and introduce other techniques we used (§3.3).

3.1 ENABLING SCALABLE REWARD UPDATE WITH IMPLICIT REWARD MODELING

We consider dense rewards from the Implicit PRM because of the scalability. In short, Implicit PRM enables training an ORM with outcome labels only while repurposing it as a PRM at inference. The training stage is the same as standard ORM pipelines, with the only difference being representing the reward as $r_\phi(\mathbf{y}) := \beta \log \frac{\pi_\phi(\mathbf{y})}{\pi_{\text{ref}}(\mathbf{y})}$, where π_ϕ is the RM and π_{ref} is the reference model, both of which are causal LMs. At inference, the process rewards are obtained by:

$$r_\phi(y_t) := \beta \log \frac{\pi_\phi(y_t|\mathbf{y}_{<t})}{\pi_{\text{ref}}(y_t|\mathbf{y}_{<t})} \tag{3}$$

In PRIME, upon rollouts being generated and graded by the (ground truth) outcome verifier, we **update the Implicit PRM online with on-policy rollouts and outcome supervision** and then **calculate token-level dense rewards to estimate advantages**, which solves C1 and C2 mentioned in §2.2 respectively: (1) To prevent overoptimization and reward hacking, it is crucial to update reward models online. However, updating previous PRMs (Lightman et al., 2023) requires annotating step labels on the latest policy rollouts, which is neither efficient nor scalable during online RL. In contrast, the Implicit PRM only demands outcome labels to train due to its special reward representation, and thus it can be easily updated with policy rollouts and outcome labels or rewards, both of which have already been collected to update the policy model. (2) Unlike common PRMs that produce only step-level rewards, the Implicit PRM provides more fine-grained *token-level* rewards at no additional cost. This addresses the ambiguity in identifying steps in LLM responses while not introducing extra overhead, making it easy to combine with any RL algorithms for advantage estimation.

Algorithm 1 Process Reinforcement through Implicit Rewards (PRIME)

Input Language model $\pi_{\theta_{\text{init}}}$; outcome reward verifier r_o; dataset $\mathcal{D}$; sample number K; total iteration N.

Initialize policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$, $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta_{\text{init}}}$, implicit PRM $\pi_\phi \leftarrow \pi_{\theta_{\text{init}}}$, reference model $\pi_{\text{ref}} \leftarrow \pi_{\theta_{\text{init}}}$
for iteration = 1, ..., N **do**
  Sample batch of prompts $\mathcal{B} \sim \mathcal{D}$
  Generate K responses: $\{\mathbf{y}^1, ..., \mathbf{y}^K\} \sim \pi_\theta(\cdot|\mathbf{x})$ for $\mathbf{x} \in \mathcal{B}$
  Compute outcome rewards: $r_o\left(\mathbf{y}^{1:K}\right)$
  Apply accuracy filter (§3.3) on all prompts: $\mathcal{T} \leftarrow \text{Filter}(\mathbf{x}, \mathbf{y}^{1:K}, r_o\left(\mathbf{y}^{1:K}\right))$ for $\mathbf{x} \in \mathcal{B}$
  Forward pass $\pi_\phi, \pi_{\text{ref}}$ on each $(\mathbf{x}, \mathbf{y}) \in \mathcal{T}$ to obatin implicit process reward $r_\phi(y_t)$ with Eq. 3
  Update Implicit PRM π_ϕ by CE loss on $(\mathbf{x}, \mathbf{y}, r_o(\mathbf{y})) \in \mathcal{T}$:

$$\mathcal{L}_{\text{CE}}(\phi) = -\mathbb{E}_{(\mathbf{x},\mathbf{y},r_o(\mathbf{y}))\sim\mathcal{T}}\left[r_o(\mathbf{y}) \cdot \log \sigma\left(r_\phi(\mathbf{y})\right) + \left(1 - r_o(\mathbf{y})\right) \cdot \log\left(1 - \sigma\left(r_\phi(\mathbf{y})\right)\right)\right]$$

  Compute advantages A with Eq. 5
  Update policy π_θ by PPO loss in Eq. 6
  Update old parameters: $\theta_{\text{old}} \leftarrow \theta$
end for

Output Optimized policy model π_θ

3.2 Advantage Estimation and Policy Update

Estimating advantages using Monte Carlo estimator with a leave-one-out baseline. After obtaining token-level dense rewards, we calculate advantages based on either MC estimators or GAE. To determine the advantage function in PRIME, we compare GAE with several MC estimators, including REINFORCE (Williams, 1992), RLOO (Ahmadian et al., 2024), and GRPO (Shao et al., 2024). Experimental details and results can be found in §5.4.

We find that MC estimators, despite being simpler, are strong enough to produce stable results. Therefore, we choose MC estimate as our advantage function and despite PRIME being compatible with any baseline estimation approaches, we instantiate it with a leave-one-out baseline from K samples (Ahmadian et al., 2024) in this paper, as it performs better in the experiments:

$$A^i = r(\mathbf{y}_T^i) - \frac{1}{K-1}\sum_{j\neq i} r(\mathbf{y}_T^j) \quad (4)$$

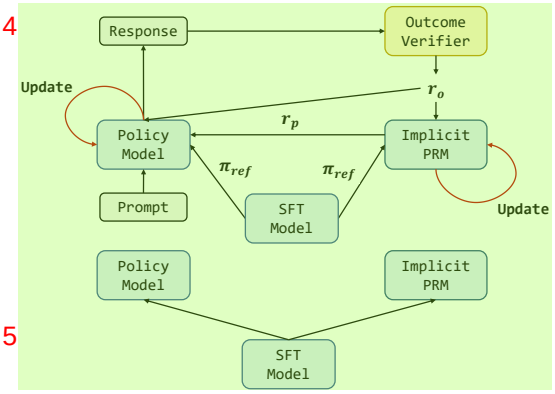


Figure 2: Illustration of PRIME. PRIME follows that (1) initialize policy model and the Implicit PRM both with the reference model; (2) sample multiple responses for each prompt and filter with output accuracy; (3) obtain implicit process rewards by the Implicit PRM and update it using cross-entropy (CE) loss; (4) compute advantage and policy loss then update the policy model.

where $r(\mathbf{y}_T^i)$ denotes the reward of i-th response at final step T, K is the number of samples for one prompt. The leave-one-out (LOO) baseline helps reduce variances.

More specifically, we use an Implicit PRM π_ϕ and an outcome verifier or reward model r_o. We calculate the return of implicit process rewards and outcome rewards separately if both are available, since directly mixing their values may lead to numerical instability (Shao et al., 2024). **For implicit process rewards**, we perform a three-step process to calculate return: (1) Use the averaged implicit process rewards to calculate the leave-one-out baseline; (2) Normalize the process reward at step t by subtracting the baseline; (3) Calculate the discounted return for each response. **For outcome rewards**, we directly adopt LOO without any modification. Finally, the advantage is set to the combination of

both returns:

$$A_t^i = \underbrace{\sum_{s=t}^{|\mathbf{y}^i|} \gamma^{s-t} \cdot \left[r_\phi(y_s^i) - \frac{1}{K-1} \sum_{j \neq i} r_\phi\left(\mathbf{y}^j\right) \right]}_{\text{RLOO with implicit process rewards}} + \underbrace{r_o\left(\mathbf{y}^i\right) - \frac{1}{K-1} \sum_{j \neq i} r_o\left(\mathbf{y}^j\right)}_{\text{RLOO with outcome rewards}} \tag{5}$$

Updating policy with PPO clip surrogate loss. We adopt PPO clip surrogate loss for more stable policy updates:

$$L_{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min\left(\frac{\pi_\theta(y_t|\mathbf{y}_{<t})}{\pi_{\theta_{\text{old}}}(y_t|\mathbf{y}_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(y_t|\mathbf{y}_{<t})}{\pi_{\theta_{\text{old}}}(y_t|\mathbf{y}_{<t})}, 1-\epsilon, 1+\epsilon \right) A_t \right) \right] \tag{6}$$

where ϵ is a clipping parameter. The loss prevents the updated policy from deviating too far from the original distribution, which is the prerequisite of importance sampling. The legitimacy of importance sampling then enables the reuse of rollouts sampled in previous steps, thus improving sampling efficiency.

3.3 Other Techniques

Initializing PRM with SFT/base model. In practice, we find that the starting policy model itself serves as a decent initialization of PRM, bypassing the PRM training stage. This solves C3 in §2.2 and even outperforms a dedicatedly trained PRM, as shown in § 5.1.

Online Prompt Filtering. As we sample multiple trajectories for each prompt, we introduce online prompt filtering which filters prompts within a certain accuracy range. This (1) preserves only the prompts within a certain median-level difficulty range (Yang et al., 2024b) and (2) balances data distribution for the Implicit PRM online training.

We present the ablation study results in Figure 3 using RLOO with outcome rewards only, from which we can see that the online prompt filter largely lowers the variance of RL training.

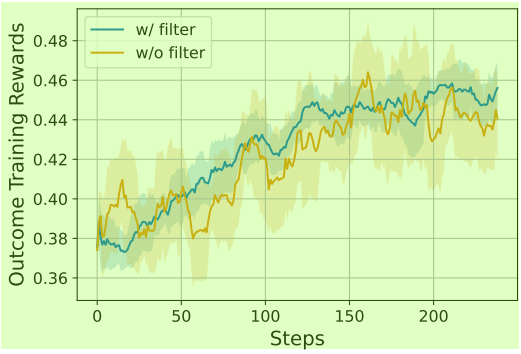


Figure 3: Impact of online prompt filtering on training rewards.

How PRIME addresses challenges in §2.2. In summary, as illustrated in Figure 2 and Algorithm 1, PRIME adopts implicit process rewards for efficient PRM online update (C2), then integrates token-level dense rewards with outcome rewards in MC advantage estimate (C1). The PRMs are directly initialized from SFT or base models, which foregoes explicit reward modeling (C3).

4 Experiments

4.1 Imitation Warmup

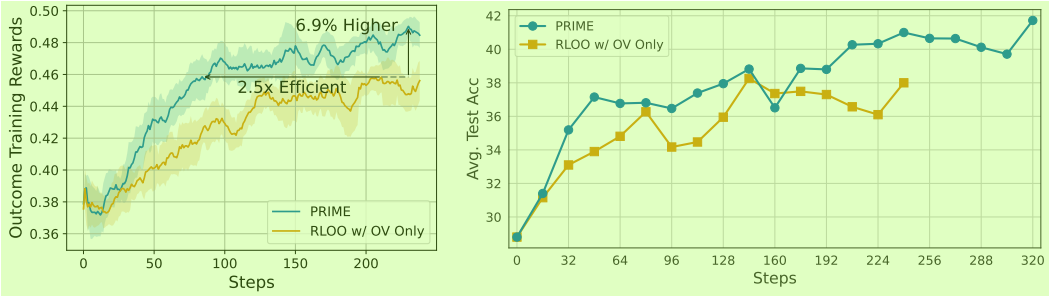
We focus on mathematical and coding problems in this paper. For models, we start with Qwen2.5-Math-7B-Base (Yang et al., 2024b) for its great mathematical capabilities. We first performed supervised finetuning for RL preparation.

Data Construction. To construct the SFT dataset, we collect reasoning instructions from several open-source datasets. For completion, we employed LLaMA-3.1-70B-Instruct (Meta, 2024) to answer the instructions, with a system prompt requesting the model to perform action-centric chain-of-thought. We finally obtained 230K SFT data, the detailed sources and statistics can be found in § A.

SFT Results. After finetuning, the performance of our SFT model is reported in Figure 1. Compared to baselines, Eurus-2-7B-SFT lags Qwen2.5-Math-7B-Instruct on all mathematics benchmarks.

Table 2: Detailed results of PRIME and RLOO w/ outcome verifier (OV). At the same 240 steps, the model trained by PRIME is generally better than the model trained by outcome rewards.

Method	Step	AIME 2024	AMC	MATH-500	MinervaMath	OlympiadBench	LeetCode	LiveCodeBench	Avg.
GPT-4o	-	9.3	45.8	76.4	36.8	43.3	58.9	48.8	45.6
Llama-3.1-70B-Inst.	-	20.0	37.3	65.0	37.1	30.5	35.0	34.4	37.0
Qwen2.5-Math-7B-Inst.	-	13.3	50.6	79.8	34.6	40.7	11.7	11.3	34.6
Eurus-2-7B-SFT	0	3.3	30.1	66.2	32.7	29.8	21.7	17.8	28.8
RLOO w/ OV Only	240	**20.0**	47.0	73.2	36.4	35.4	28.3	26.7	36.9
Eurus-2-7B-PRIME	80	20.0	41.0	68.2	38.2	37.0	26.7	26.6	36.8
	160	13.3	42.2	72.0	37.1	38.7	26.7	25.6	36.5
	240	**20.0**	**50.6**	**78.2**	**39.3**	**40.3**	**31.1**	**27.5**	**41.0**
	320	16.7	51.8	77.8	39.7	41.5	36.1	28.5	41.7
	592	26.7	57.8	79.2	38.6	42.1	33.3	28.6	43.9



(a) Outcome training rewards (10-step moving). (b) Test accuracy across different gradient steps.

Figure 4: **The effect of dense reward.** We compare PRIME and RLOO with outcome verifier (OV). Dense rewards in PRIME lead to 2.5× sample efficiency and 6.9% performance improvement. PRIME also substantially outperforms RLOO on downstream tasks.

4.2 RL Settings

Rule-based Outcome Verifier. Consistent with recent research that adopts exact match with ground truth as unhackable rewards (Gao et al., 2024; Lambert et al., 2024; DeepSeek-AI et al., 2025), we define the rule-based ground truth outcome verifiers (OV) for math and coding as follows:

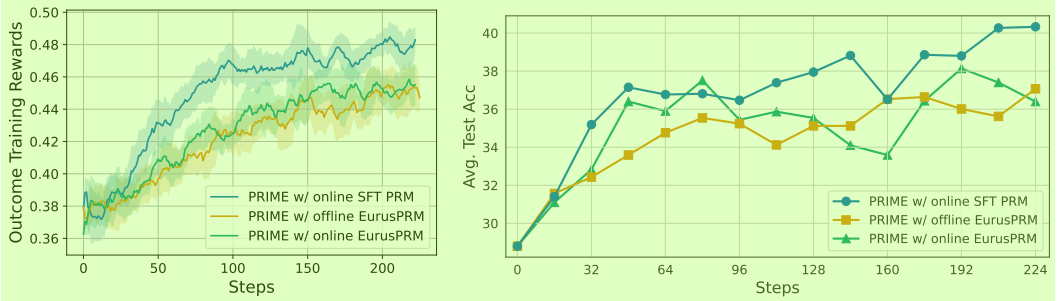
$$r_o^{\text{math}}(\mathbf{y}) = \begin{cases} 1, & \text{matched} \\ 0, & \text{otherwise} \end{cases} \qquad r_o^{\text{code}}(\mathbf{y}) = \frac{\sum \#\text{passes}}{\sum \#\text{test cases}}$$

Hyperparameters. We use veRL (Sheng et al., 2024) to conduct experiments. By default, we initialize the Implicit PRM with SFT model and retain the SFT model for reference logprobs. For hyperparameters, we use a constant 5×10^{-7} learning rate together with AdamW optimizer for policy model, and use a 10^{-6} learning rate for PRMs. Both policy and PRMs use a batch size of 256 and micro batchsize of 8. The rollout stage collects 256 prompts and samples 4 responses for each prompt. We set $\beta = 0.05$ for PRM training. We set KL coefficient to 0 in all experiments.

Evaluation Benchmarks. We evaluate on 7 reasoning benchmarks, focusing on competition-level mathematics and programming tasks, including AIME 2024 (Li et al., 2024), AMC (Li et al., 2024), MATH-500 (Hendrycks et al., 2021b), Minerva Math (Lewkowycz et al., 2022), OlympiadBench (He et al., 2024), LeetCode (Guo et al., 2024), and LiveCodeBench (v2) (Jain et al., 2024).

4.3 Main Results

As shown in Figure 1 and Table 2, Eurus-2-7B-PRIME achieves substantial improvements on key reasoning benchmarks over the SFT version of the model, leading to 15.1% improvement on average, and over 20% on AMC and AIME competitions. Besides, Eurus-2-7B-PRIME achieves 26.7% pass@1 on AIME 2024, surpassing GPT-4o, Llama-3.1-70B-Instruct, and Qwen2.5-Math-7B-Instruct, demonstrating its excellent reasoning ability.



(a) Outcome training rewards (10-step moving). (b) Test accuracy across different gradient steps.

Figure 5: **Comparison of different PRMs.** Online PRM initialized from SFT model achieved the best results. Surprisingly, using PRMs trained on extra rollouts hurts the performance in both online and offline settings.

4.4 Dense Rewards v.s. Sparse Rewards

We first validate the effect of dense rewards compared to RLOO with outcome rewards only. We train this model for 240 steps. For PRIME, we use the same setting and train the model for 592 steps. We plot the training rewards measured by the outcome verifier and test accuracy in Figure 4. **Compared with sparse reward, PRIME takes 40% of the training steps to achieve the same training rewards as RLOO and improves the final rewards by 6.9%, with lower variances.** On downstream tasks, PRIME also consistently outperforms OV only setup. Detailed results are listed in Table 2.

5 Analysis

5.1 Design Choices for the Implicit PRM

The Implicit PRM is the key component of PRIME, and its design choices greatly affect RL. In this section, we explore two major factors: (1) the initialization model and (2) the update mechanism.

SFT model initializes a good PRM. Conventionally, we need to collect data to train RMs and PRMs, and then we can use them in RL. However, the Implicit PRM is a language model, so we can initialize it from any language model with the same tokenizer as the policy model. To investigate whether it is still necessary to train a PRM in advance, we conduct experiments with different PRM initialization strategies: with the SFT model itself and with a specially trained PRM. For the later one, we train EurusPRM from Eurus-2-7B-SFT with additional 500K data generated by Llama3.1 and Qwen2.5 series (data details in § B.5).

We report the experiment results in Figure 5. **Surprisingly, directly using Eurus-2-7B-SFT to initialize the PRM greatly outperforms EurusPRM which was trained on more samples.** We conjecture that initializing policy model and PRM from the same model largely alleviates the distribution shift issue, as the PRM is only trained on the online rollouts from the policy model.

Online PRM update is essential. To verify the effect of online PRM update, we pair the correct and wrong samples and calculate the PRM prediction accuracy using $r_\phi(\mathbf{y})$. We report the PRM classification accuracy in Figure 6. The figure clearly shows that, **online update mitigates overoptimization and reward**

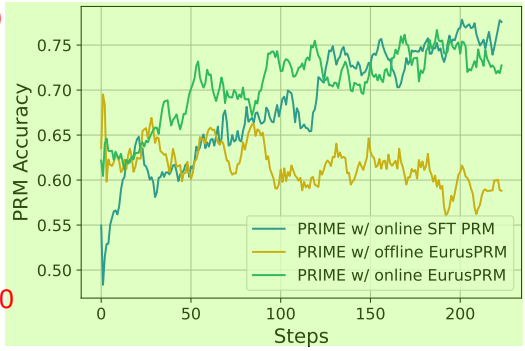
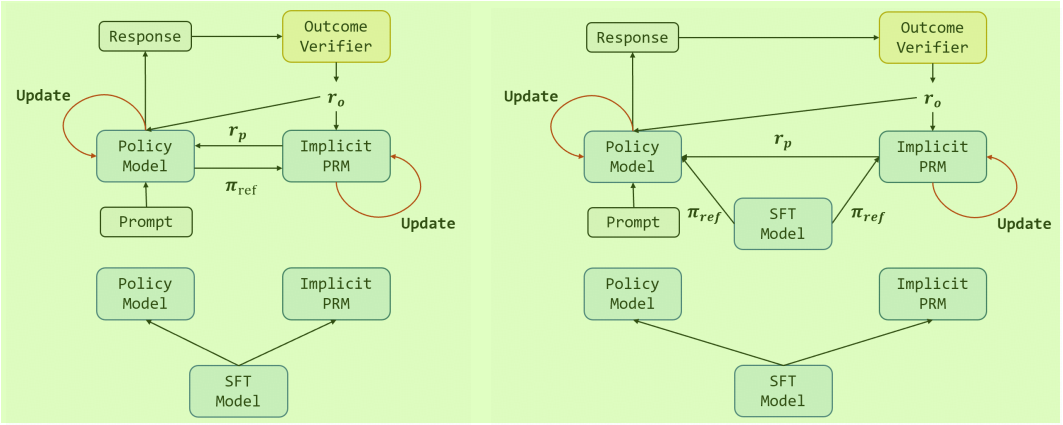


Figure 6: **Impact of PRM online update.** The offline PRM is gradully been overoptimized while online PRMs achieve higher accuracy throughout training.



(a) Policy ref: We use the policy logprob as π_{ref} for PRM.

(b) SFT ref: We retain the initial policy to provide π_{ref} for PRM and KL.

Figure 7: Comparison of different reference policy implementations. One uses the running policy's old logprobs as reference (policy ref) while the other uses the initial SFT model as the reference model (SFT ref).

hacking. The offline PRM, though starting with high accuracy, gradually drops during RL training procedure due to distribution shift. In contrast, online PRMs that are trained on policy rollouts show the reverse curve.

This is further validated with training rewards and downstream performance. To breakdown, Eurus-2-7B-SFT is both used as PRM initialization and the reference model in the main experiment, so the PRM is totally trained from scratch, which means the initial PRM outputs zero reward for all tokens. Therefore, Figure 4 also demonstrates the effect of online PRM update. For EurusPRM initialization, the online run outperforms the offline run as well in Figure 5.

5.2 Reference Model Choice is Flexible

We implement two variants of our algorithms to explore the effect of reference model of implicit PRM, one using the initial SFT model as the reference model (SFT ref) while the other using the running policy's old logprobs as reference (policy ref), as shown in Figure 7a. The policy ref simply adopts the old logprob of the policy model as π_{ref}, while the SFT ref remains the initial SFT model for an additional π_{ref} calculation. We compare their performance in this section.

From the training rewards in Figure 8, we find the two strategies are close and have pros and cons in different aspects: The Q value calculated by implicit PRM is the expectation under the distribution of the reference model. So the updating policy could natrually serve as the reference. On the other hand, KL divergence calculation is only allowed when the initial SFT model is retained.

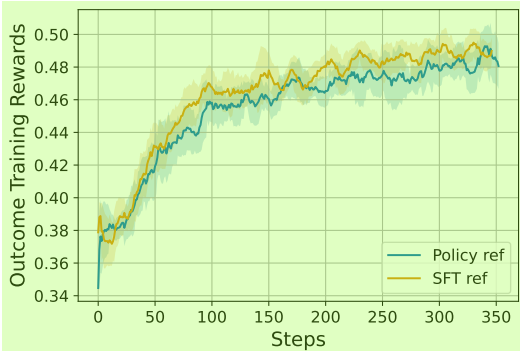


Figure 8: **Different reference model for PRM.** We compare two reference model selection strategies for PRIME. Using the policy model as reference and using the initial SFT model as reference. Their rewards are similar.

5.3 Single-Forward v.s. Double-Forward

Since our implicit PRM is concurrently updated in training, for each rollout stage, we can update the PRM before the policy model and use the updated PRM to re-calculate the process rewards, which

(a) PRM classification accuracy on training samples. (b) Training outcome rewards.

Figure 9: **Single and double forward.** While double forward methods obtain higher accuracy after online update, the two variants achieve similar rewards during training.

we call the double-forward setting. We investigate the impact of double-forward in both the training and test phases. Our default setting applies single-forward, which uses process rewards from old PRMs. We plot PRM accuracy on rollouts and training rewards in Figure 9.

Accordingly, we find that double-forward could increase PRM accuracy, but the training rewards remain close between the two methods.

5.4 PRIME WITH OTHER RL ALGORITHMS

As we stated before, PRIME is equally applicable to other RL algorithms beyond RLOO. In this section, we implement PRIME with REINFORCE (Williams, 1992), GRPO (Shao et al., 2024), and PPO (Schulman et al., 2017). Similarly to RLOO, we only modify the advantage estimation functions and leave the clip surrogate loss unchanged.

First of all, We compare different REINFORCE-like advantage estimators including REINFORCE, GRPO, and RLOO, toggling the existence of implicit process reward. To make different algorithms compatible with the compound of outcome verifier reward and process reward, we accordingly make adaptions similar to Eq. 5. For GRPO, we have

$$A_t^i = \underbrace{\frac{r_o\left(\mathbf{y}^i\right) - \text{mean}(r_o\left(\mathbf{y}^j\right))}{\text{std}(r_o\left(\mathbf{y}^j\right))}}_{\text{GRPO with outcome rewards}} + \underbrace{\sum_{s=t}^{|\mathbf{y}^i|} \gamma^{s-t} \cdot \left[\frac{r_\phi(y_s^i) - \text{mean}\left(\frac{r_\phi(\mathbf{y}^j)}{|\mathbf{y}^j|}\right)}{\text{std}\left(\frac{r_\phi(\mathbf{y}^j)}{|\mathbf{y}^j|}\right)}\right]}_{\text{GRPO with implicit process rewards}}. \tag{7}$$

For REINFORCE, we have

$$A_t^i = \underbrace{r_o\left(\mathbf{y}^i\right)}_{\text{REINFORCE with outcome rewards}} + \underbrace{\sum_{s=t}^{|\mathbf{y}^i|} \gamma^{s-t} \cdot r_\phi(y_s^i)}_{\text{REINFORCE with implicit process rewards}}. \tag{8}$$

From Figure 10 and Table 3, We show that PRIME boosts these algorithms on both efficiency and performance as it does with RLOO. PRIME contributes consistently regardless of the policy update method, making it a generic algorithm. It indicates that **PRIME is a general plug-in for almost any RL algorithm for LLM.**, which largely extends the use cases of PRIME.

Moreover, the PPO variant of PRIME provides no performance gain, demonstrating that the additional computation cost from the critic model is redundant. This makes it possible to compensate for the expense of the process reward model by using REINFORCE-like algorithms with simpler advantage estimators. Finally, we choose the best-performing RLOO as the advantage estimator in our algorithm.

Table 3: Testset results of different RL algorithms.

Method	Step	AIME 2024	AMC	MATH-500	MinervaMath	OlympiadBench	LeetCode	LiveCodeBench	Avg.
RLOO	240	20.0	47.0	73.2	36.4	35.4	28.3	26.7	36.9
RLOO w/ PRIME	240	20.0	50.6	78.2	39.3	40.3	31.1	27.5	41.0
REINFORCE	240	6.7	47.0	72.6	36.0	37.2	27.2	25.0	36.0
REINFORCE w/ PRIME	240	6.7	50.0	76.4	36.8	39.1	27.8	27.5	37.8
GRPO	240	10.0	44.6	73.2	37.5	36.6	25.0	25.8	36.1
GRPO w/ PRIME	240	16.7	47.0	75.0	34.9	38.2	28.9	23.9	37.8
PPO	240	10.0	41.0	73.6	36.0	36.3	28.3	25.7	35.8
PRIME as Value Model	240	16.7	44.6	72.6	34.6	35.7	27.8	24.6	36.6
PPO w/ PRIME	240	13.3	50.6	77.4	37.1	40.6	30.0	26.7	39.4

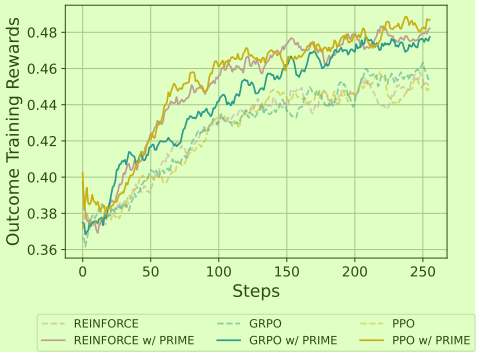


Figure 10: PRIME also benefits REINFORCE, GRPO, and PPO, achieving similar improvement as RLOO.

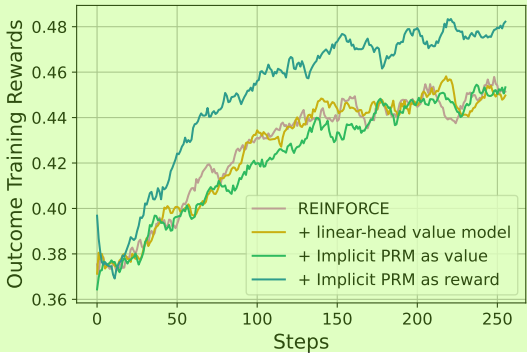


Figure 11: Comparison of value models and reward models. We show that value models, either the original PPO one or Implicit PRM, is substaintially worse than reward models.

5.5 Value or Reward, How to Use the Implicit PRM?

Besides using process rewards to estimate returns, we can also employ the Implicit PRM to predict values for advantage estimation in Eq. 2. Therefore, we compare four variants of MC estimate to determine the best way to incorporate dense supervision. Recall that the Implicit PRM has $v_\phi(\mathbf{y}_{<t+1}) = \sum_{i=1}^{t} \beta \log \frac{\pi_\phi(y_i|\mathbf{y}_{<i})}{\pi_{\text{ref}}(y_i|\mathbf{y}_{<i})}$ with the process reward being $r_\phi(y_t) = v_\phi(\mathbf{y}_{<t+1}) - v_\phi(\mathbf{y}_{<t})$, and we assume a ground-truth outcome verifier r_o, $\gamma = 1$, then we represent the variants as follows:

(1) REINFORCE: $A_t = r_o(\mathbf{y})$.

(2) On top of (1), using **a linear-head value model** V to calculate the baseline: $A_t = r_o(\mathbf{y}) - V(\mathbf{y}_{<t})$. This is the original PPO in Figure 10 as we set $\gamma = 1$ and $\lambda = 1$.

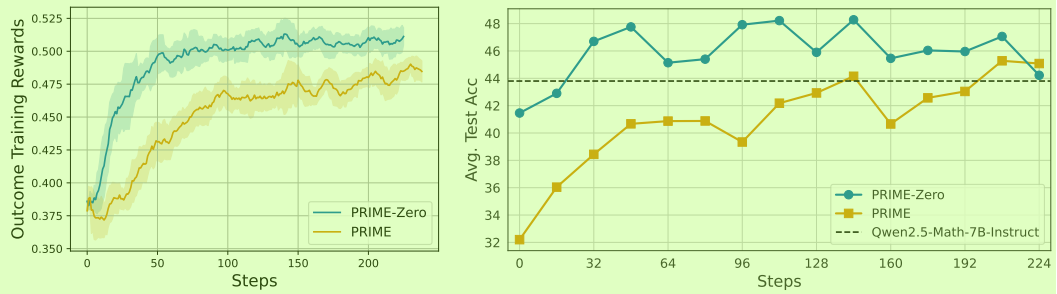
(3) On top of (1), using **values from the Implicit PRM** to serve as the baseline: $A_t = r_o(\mathbf{y}) - v_\phi(\mathbf{y}_{<t})$. This is equivalent to PPO with its value model being replaced by values from the Implicit PRM when $\gamma = 1$ and $\lambda = 1$.

(4) On top of (1), using **process rewards from the Implicit PRM** to calculate the return: $A_t = r_o(\mathbf{y}) + \sum_{s=t}^{T} r_\phi(y_s)$. This is the REINFORCE w/ PRIME in Figure 10.

Figure 11 reports the results. Comparing PPO and REINFORCE, we find that an additional value model does not benefit policy performance. Notably, using rewards from the Implicit PRM to calculate returns, which is the default setting in PRIME, greatly outperforms all three baselines, regardless of where the values come from. This indicates that PRMs work better than value models in RL for LLMs.

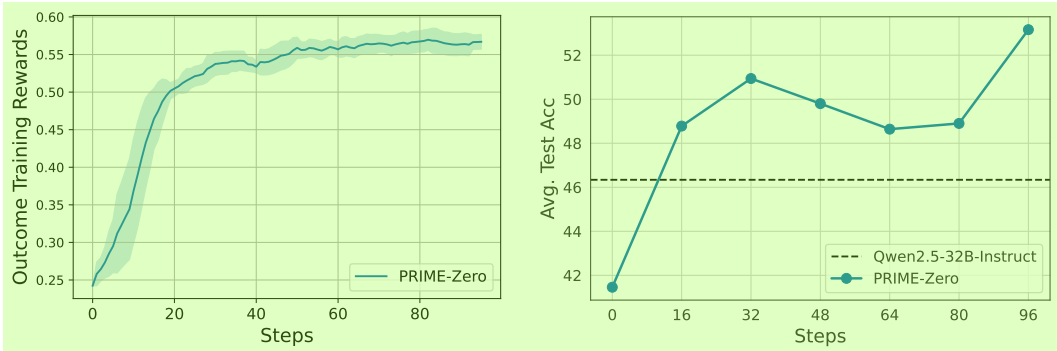
5.6 "Zero" Experiments

DeepSeek-AI et al. (2025) proposed DeepSeek-R1-Zero, which is directly trained from a base model with reinforcement learning. To further investigate the "Zero" setting, we also perform RL from



(a) Outcome training rewards (10-step moving). (b) Math test accuracy across different gradient steps.

Figure 12: **"Zero" RL from Qwen2.5-Math-7B.** RL from the base model converges way faster than the SFT model, surpassing the instruct version within 32 steps.



(a) Outcome training rewards (10-step moving). (b) Math test accuracy across different gradient steps.

Figure 13: **"Zero" RL from Qwen2.5-32B-Base.** RL from a 32B base model shows more promising gain, surpassing the instruct version within 16 steps.

Qwen2.5-Math-7B-Base and Qwen2.5-32B-Base (Yang et al., 2024a), skipping the SFT phase. We present the experimental results in Figure 12 and Figure 13. The observations are as follows:

(1) **RL from base model is suprisingly efficient and effective.** Comparing PRIME from Qwen2.5-Math-7B and Eurus-2-7B-SFT, the "Zero" setting converges much faster. This indicates that directly performing RL from a base model might be a strong alternative to the conventional SFT-RL pipeline.

(2) **Larger models benefit more.** Comparing 7B and 32B models, we see that the 32B model gains more on both training rewards and test performance. This is aligned with the conclusion in DeepSeek-AI et al. (2025).

(3) **Saturation could be a potential issue.** Although PRIME-Zero obtains impressive performance gain, we find it quickly saturated at a very early stage (about 50 steps), which hinders further improvement like in DeepSeek-AI et al. (2025). This is possibly attributed to the decrease of response diversity, and we leave this as future work.

6 Related Work

RL for LLM Reasoning. In the context of LLMs, reinforcement learning has been widely used for aligning human preferences (Christiano et al., 2017; Ouyang et al., 2022; Cui et al., 2024), but the open-source community mostly adopt the data-driven imitation learning methods (Yuan et al., 2024a; Yue et al., 2024; Wei et al., 2024; Liu et al., 2024) to enhance the reasoning capabilities of LLMs. Over the past few months, the paradigm gradually shifted. OpenAI o1 (Jaech et al., 2024) first showed the tremendous potential of large-sacle RL for reasoning LLMs, and recent works have verified the scaling effect of the simple RL recipe with merely outcome rewards (DeepSeek-AI et al., 2025; Team

et al., 2025). Meanwhile, the role of dense rewards in RL remains underexplored, which is the main focus of PRIME.

Implicit Rewards. Implicit rewards are broadly adopted in LLM alignment (Rafailov et al., 2023; Chen et al., 2024b; Azar et al., 2024; Ethayarajh et al., 2024; Rosset et al., 2024; Chen et al., 2024a). Rafailov et al. (2024) first showed that optimizing DPO objective learns a Q function implicitly. Zhou et al. (2024) utilized implicit rewards in PPO, and showed that dense implicit rewards are better than sparse ones. Yuan et al. (2024b) further extended the conclusion to any loss funtion optimizing Eq. 3.

7 Conclusion

As the fuel of LLMs, data, will be depleted in the near future, we are entering a new era of search and exploration, which is exemplified by reinforcement learning (Sutton, 2019). This work develops PRIME, which produces and leverages dense rewards in online RL for LLM reasoning. Throughout the experiments, we validate that PRIME (1) greatly benefits sample efficiency and policy performance, (2) is easy to use with minimum cost, and (3) is a general method that works with broad RL algorithms together.

References

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. *arXiv preprint arXiv:2402.14740*, 2024.

Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A general theoretical paradigm to understand learning from human preferences. *International Conference on Artificial Intelligence and Statistics*, abs/2310.12036, 2024.

Changyu Chen, Zichen Liu, Chao Du, Tianyu Pang, Qian Liu, Arunesh Sinha, Pradeep Varakantham, and Min Lin. Bootstrapping language models with dpo implicit rewards. *arXiv preprint arXiv:2406.09760*, 2024a.

Huayu Chen, Guande He, Lifan Yuan, Ganqu Cui, Hang Su, and Jun Zhu. Noise contrastive alignment of language models with explicit rewards. *arXiv preprint arXiv:2402.05369*, 2024b.

Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.

Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, Zhiyuan Liu, and Maosong Sun. Ultrafeedback: Boosting language models with scaled ai feedback. In *ICML*, 2024.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng

Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948. 1

Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. Kto: Model alignment as prospect theoretic optimization. *ICML*, 2024. 2

Jiaxuan Gao, Shusheng Xu, Wenjie Ye, Weiling Liu, Chuyi He, Wei Fu, Zhiyu Mei, Guangju Wang, and Yi Wu. On designing effective rl reward at training time for llm reasoning. *ArXiv*, abs/2410.15115, 2024. 3

Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning*, 2022. 4

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024. 5

Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3828–3850, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.211. URL https://aclanthology.org/2024.acl-long.211/. 6

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021a. 7

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021b. 8

Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024. 9

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024. 10

Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. Vineppo: Unlocking rl potential for llm reasoning through refined credit assignment. *arXiv preprint arXiv:2410.01679*, 2024. 11

Wouter Kool, Herke van Hoof, and Max Welling. Buy 4 reinforce samples, get a baseline for free! In *DeepRLStructPred@ICLR*, 2019. URL https://api.semanticscholar.org/CorpusID:198489118. 12

Nathan Lambert, Jacob Daniel Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James Validad Miranda, Alisa Liu, Nouha Dziri, Xinxi Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hanna Hajishirzi. Tülu 3: Pushing frontiers in open language model post-training. *ArXiv*, abs/2411.15124, 2024. 1

Jan Leike, David Krueger, Tom Everitt, Miljan Martic, Vishal Maini, and Shane Legg. Scalable agent alignment via reward modeling: a research direction. *arXiv preprint arXiv:1811.07871*, 2018. 2

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *Advances in Neural Information Processing Systems*, 35:3843–3857, 2022. 3

Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q Jiang, Ziju Shen, et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. *Hugging Face repository*, 13:9, 2024. 4

Rongao Li, Jie Fu, Bo-Wen Zhang, Tao Huang, Zhihong Sun, Chen Lyu, Guang Liu, Zhi Jin, and Ge Li. Taco: Topics in algorithmic code generation dataset. *arXiv preprint arXiv:2312.14852*, 2023. 5

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *arXiv preprint arXiv:2203.07814*, 2022. 6

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *ArXiv*, abs/2305.20050, 2023. 7

Zihan Liu, Yang Chen, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. Acemath: Advancing frontier math reasoning with post-training and reward modeling. *arXiv preprint arXiv:2412.15084*, 2024. 8

Meta. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783. 9

OpenAI. Openai o1 system card. *ArXiv*, abs/2412.16720, 2024. 10

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022. 11

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2023. 12

Rafael Rafailov, Joey Hejna, Ryan Park, and Chelsea Finn. From r to q^*: Your language model is secretly a q-function. *arXiv preprint arXiv:2404.12358*, 2024. 13

Corby Rosset, Ching-An Cheng, Arindam Mitra, Michael Santacroce, Ahmed Awadallah, and Tengyang Xie. Direct nash optimization: Teaching language models to self-improve with general preferences. *ArXiv*, abs/2404.03715, 2024. 14

John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016. 15

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 1

Amrith Setlur, Chirag Nagpal, Adam Fisch, Xinyang Geng, Jacob Eisenstein, Rishabh Agarwal, Alekh Agarwal, Jonathan Berant, and Aviral Kumar. Rewarding progress: Scaling automated process verifiers for llm reasoning. *arXiv preprint arXiv:2410.08146*, 2024. 2

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300. 3

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024. 4

SkunkworksAI. reasoning-0.01, 2024. 5

Richard Sutton. The bitter lesson. *Incomplete Ideas (blog)*, 13(1):38, 2019. 6

Richard S Sutton. Learning to predict by the methods of temporal differences. *Machine learning*, 3: 9–44, 1988. 7

Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018. 8

Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025. 9

Qwen Team. Qwq: Reflect deeply on the boundaries of the unknown, November 2024. URL https://qwenlm.github.io/blog/qwq-32b-preview/. 10

Shubham Toshniwal, Wei Du, Ivan Moshkov, Branislav Kisacanin, Alexan Ayrapetyan, and Igor Gitman. Openmathinstruct-2: Accelerating ai for math with massive open-source instruction data. *arXiv preprint arXiv:2410.01560*, 2024. 11

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022. 12

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Y.Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. *ArXiv*, abs/2312.08935, 2023. 13

Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Empowering code generation with oss-instruct. In *Forty-first International Conference on Machine Learning*, 2024. 14

Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229–256, 1992. 15

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024a. 16

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement, 2024b. URL https://arxiv.org/abs/2409.12122. 17

Lifan Yuan, Ganqu Cui, Hanbin Wang, Ning Ding, Xingyao Wang, Jia Deng, Boji Shan, Huimin Chen, Ruobing Xie, Yankai Lin, Zhenghao Liu, Bowen Zhou, Hao Peng, Zhiyuan Liu, and Maosong Sun. Advancing llm reasoning generalists with preference trees. *ArXiv*, 2024a. 1

Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. Free process rewards without process labels, 2024b. URL https://arxiv.org/abs/2412.01981. 2

Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. Mammoth: Building math generalist models through hybrid instruction tuning. *arXiv preprint arXiv:2309.05653*, 2023. 3

Xiang Yue, Tuney Zheng, Ge Zhang, and Wenhu Chen. Mammoth2: Scaling instructions from the web. *ArXiv*, abs/2405.03548, 2024. 4

Kaiyan Zhang, Sihang Zeng, Ermo Hua, Ning Ding, Zhang-Ren Chen, Zhiyuan Ma, Haoxin Li, Ganqu Cui, Biqing Qi, Xuekai Zhu, Xingtai Lv, Hu Jinfang, Zhiyuan Liu, and Bowen Zhou. Ultramedical: Building specialized generalists in biomedicine, 2024. 5

Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhu Chen, and Xiang Yue. Opencodeinterpreter: Integrating code generation with execution and refinement. *arXiv preprint arXiv:2402.14658*, 2024. 6

Zhanhui Zhou, Zhixuan Liu, Jie Liu, Zhichen Dong, Chao Yang, and Yu Qiao. Weak-to-strong search: Align large language models via searching over small language models. *arXiv preprint arXiv:2405.19262*, 2024. 7

Table 4: Actions in action-centric chain-of-thought reasoning framework.

Action Name	Description
ASSESS	Analyze current situation, identify key elements and goals
ADVANCE	Move forward with reasoning - calculate, conclude, or form hypothesis
VERIFY	Check accuracy of current approach, look for errors
SIMPLIFY	Break complex problems into simpler parts
SYNTHESIZE	Combine multiple pieces of information into complete solution
PIVOT	Change strategy when current approach isn't working
OUTPUT	Summarize thought process and present final answer

Table 5: Data statistics of SFT data.

Task	Dataset	Size	Avg. Response Length	Source
Math	MathInstruct-MATH (Yue et al., 2023)	12715	964.01	https://huggingface.co/datasets/TIGER-Lab/MathInstruct
	OpenMathIns-2-Aug_Math (Toshniwal et al., 2024)	15086	1202.25	https://huggingface.co/datasets/nvidia/OpenMathInstruct-2
	Numina (Li et al., 2024)	55845	1331.61	https://huggingface.co/datasets/AI-MO/NuminaMath-CoT
	Reasoning-001 (SkunkworksAI, 2024)	29831	1316.49	https://huggingface.co/datasets/SkunkworksAI/reasoning-0.01
Coding	Code-Feedback (Zheng et al., 2024)	27663	1805.16	https://huggingface.co/datasets/m-a-p/Code-Feedback
	Magicoder (Wei et al., 2024)	24480	1828.72	https://huggingface.co/datasets/ise-uiuc/Magicoder-Evol-Instruct-110K
	Magicoder-OSS (Wei et al., 2024)	28980	1850.05	https://huggingface.co/datasets/ise-uiuc/Magicoder-OSS-Instruct-75K
Biomedicine	UltraMedical_mc (Zhang et al., 2024)	35163	891.06	https://huggingface.co/datasets/TsinghuaC3I/UltraMedical
Total / Avg.	-	229763	1390.75	-

A SFT Data & Training Details

We first perform supervised finetuning on the base model to get a starter model for RL.

Action-centric chain-of-thought reasoning. We apply imitation learning (supervised finetuning) as a warmup stage to teach models to learn certain reasoning patterns. To this end, we first design an action-centric chain-of-thought reasoning framework. Table 4 shows the actions in the action-centric chain-of-thought reasoning framework. When the model generates answers, it conducts multi-step reasoning and chooses one of the 7 actions at each step. The response begins with the ASSESS action and ends with the OUTPUT action.

Construction of the SFT dataset. To construct the SFT dataset, we collect reasoning instructions from several open-source datasets. It is noteworthy that we did not include many datasets with ground-truth answers in SFT, even though they are of higher quality. However, we reserve them for later RL training. The reason is that we aim to use different datasets for SFT and RL to diversify the exploration in RL, and we consider ground-truth more essential in RL than in SFT. For completion, we employ LLaMA-3.1-70B-Instruct to answer the instructions, with a system prompt requesting the model to perform an action-centric chain-of-thought. Table 5 summarizes the key statistics of the datasets used for SFT. The datasets span mathematics, coding, and biomedicine. We finally obtain 230K SFT data and the average response length is 1390 tokens.

SFT Training. During the SFT phase, we conduct full parameter fine-tuning with a learning rate of 1e-05, utilizing the AdamW optimizer alongside a cosine annealing learning rate schedule and a warmup ratio of 0.1. The batch size was set to 96, with a fixed random seed of 42. The model was trained on 230K datasets for 3 epochs.

B RL Data Preprocessing

B.1 RL Data Collection and Preprocessing

We curate a high-quality RL training dataset of mathematics and coding problems with outcome verifiers (LaTeX answers for math and test cases for coding). For math, we source from NuminaMath-CoT (Li et al., 2024), which contains about 860K math problems. The problems span from Chinese high school mathematics to International Mathematical Olympiad competition questions. For coding, we source from APPS (Hendrycks et al., 2021a), CodeContests (Li et al., 2022), TACO (Li et al., 2023), and Codeforces[2]. To further increase data quality, we conduct detailed cleaning and filtering. Finally, we retain 457k math problems and 27k coding problems.

[2] https://huggingface.co/datasets/MatrixStudio/Codeforces-Python-Submissions

B.2 Data Filtering and Question-Type Classification

The preprocessing pipeline employs a systematic rule-based approach to filter and classify mathematical problems to create a high-quality dataset with solvable problems, appropriate difficulty levels, and correct solutions. We exclude problems containing figures or diagrams since they require visual processing capabilities. We also remove proof questions due to difficulties in answer verification. Based on specific patterns, the remaining problems are classified into question-answering, multiple-choice, or fill-in-the-blank questions. Since fill-in-the-blank questions comprise less than 400 examples compared to the much larger set of multiple-choice questions, we focus solely on multiple-choice questions for further processing.

B.3 Converting to Direct Question-Answer Format

We transform multiple-choice questions into a direct question-answer format through three sequential stages: rule-based filtering, LLM-based filtering, and LLM-based formatting.

We first identify and remove questions that inherently require multiple-choice options - specifically, those where comparing specific statements or properties is essential to the problem-solving process. These questions cannot be meaningfully converted to a direct question-answer format. The initial filtering employs simple rule-based pattern matching, searching for keywords like ”following” and ”statement” that typically indicate option-dependent problems.

Following the rule-based filtering, we employ Llama-3.1-8B-Instruct to perform a more nuanced classification of the remaining questions. Our pilot study revealed that while the LLM occasionally misclassifies questions, it tends to err on the conservative side - marking potentially convertible questions as requiring options rather than the reverse. Given our large dataset, we accepted this conservative approach to maintain quality.

For questions classified as convertible, we implement a two-phase reformatting process: 1) Question Reformatting: Removing choice indicators and restructuring the question to elicit direct answers. 2) Solution Reformatting: Converting multiple-choice solutions into step-by-step derivations, ensuring all final answers are presented in standard LaTeX boxed format. This systematic approach maintains mathematical rigor while creating a standardized format suitable for downstream applications.

B.4 Problem and Solution Validation

The final stage involves merging all question-answer pairs and performing LLM-based comprehensive validation. We identify two key aspects in validation: solvability and correctness.

We leverage state-of-the-art mathematical reasoning models, including QwQ-32B-Preview (Team, 2024) and Qwen2.5-Math-72B-Instruct (Yang et al., 2024b), employing a self-consistency approach to determine problem solvability, and if solvable, verify the correctness of solutions provided in the original dataset.

To enhance validation accuracy, we first analyzed sample problems to identify characteristics of solvable and unsolvable cases and created synthetic unsolvable problems featuring missing conditions or logical contradictions. Based on these samples, we developed specialized prompts to improve the models' ability to distinguish solvability. Each problem undergoes five independent validation attempts, where the LLM: 1) Provides step-by-step solutions using LaTeX formatting. 2) Identifies unsolvability due to missing conditions or logical contradictions. 3) Generates complete reasoning traces for solvable problems. 4) Presents final answers in standardized LaTeX boxed format (`\boxed{...}`). 5) Document any impediments to solution completion.

We evaluate two key consistency measures across multiple validation attempts: 1) Status Consistency: agreement on problem solvability. 2) Answer Consistency: consistency of solutions across different attempts and agreement between generated solutions and ground truth. The final dataset retains only problems that demonstrate consistent solvability across validation attempts, agreement in solutions across multiple attempts, and alignment with ground truth answers. This rigorous validation process ensures the resulting dataset comprises well-defined, solvable problems with verified, accurate solutions.

Table 6: Data statistics of EurusPRM training dataset.

Dataset	Generator Model	Num. Inst	Resp/Inst	Step-level/Response-level
UltraInteract	Llama-3.1-8B-Inst	20177	8	Response-level
	Llama-3.1-8B-Base	13570	8	Response-level
	Qwen2.5-72B-Inst	4758	8	Response-level
	Qwen2.5-Math-7B-Base	25713	8	Response-level
Numina-SynMath	Llama-3.1-8B-Inst	4783	8	Response-level
	Qwen2.5-Math-7B-Base	5806	8	Response-level
Numina-Olympiads	Llama-3.1-8B-Inst	2909	8	Response-level
	Qwen2.5-Math-7B-Base	4739	8	Response-level

B.5 PRM DATA

The dataset statistics of training EurusPRM are shown in Table 6.